EKS Pod Identity로 더 간편하게 Kubernetes 서비스 권한 관리하기

김태지 (Ethan)

번개장터 DevSecOps Engineer

AWS KRUG 마곡 DevOps 소모임 · 2026-04-28

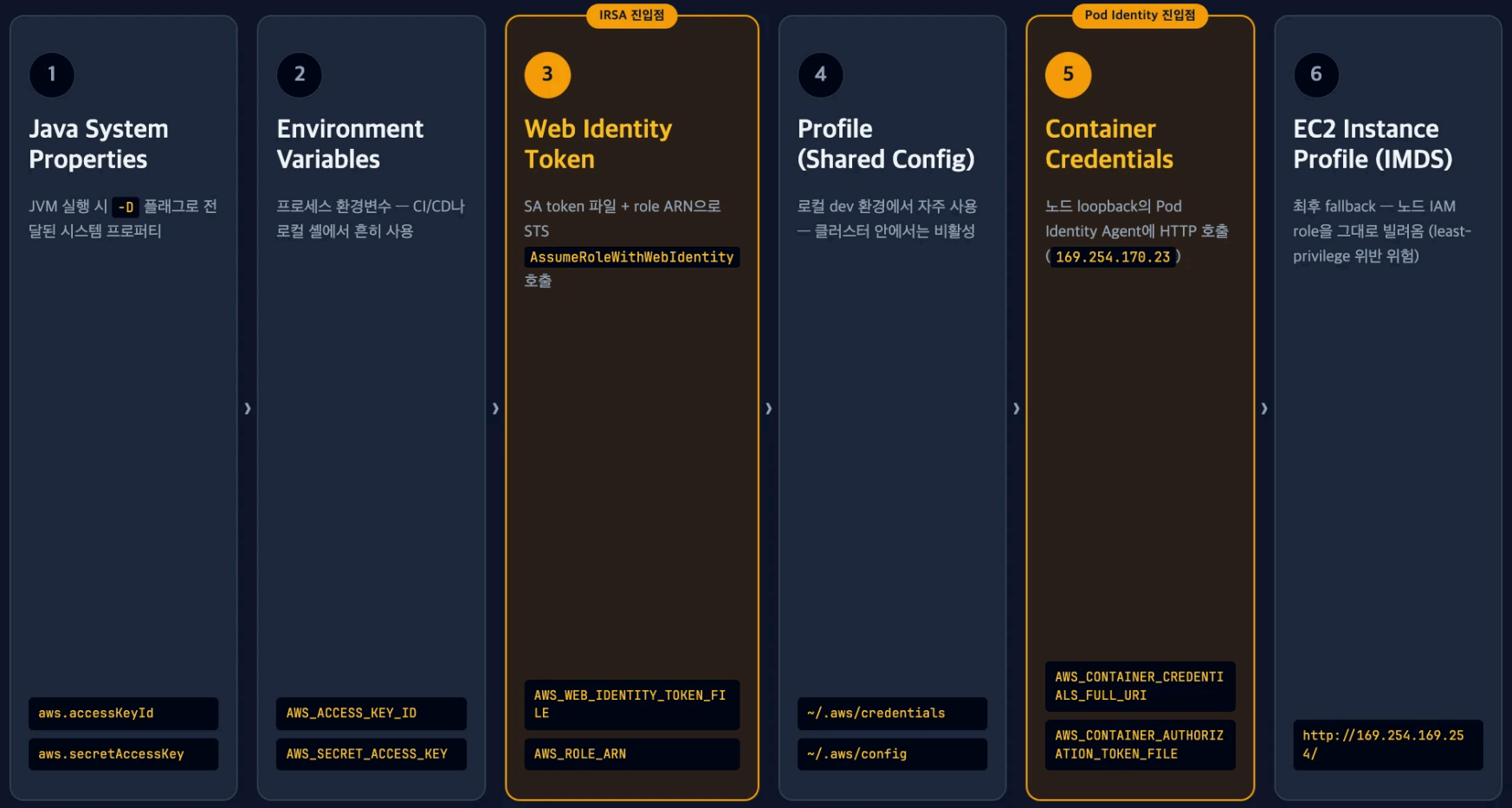
목차

1. Pod이 AWS 리소스에 접근하는 방법 — credential provider chain
2. IRSA와 Pod Identity 구조 비교
3. 멀티클러스터 환경에서 IRSA의 운영 부담
4. Pod Identity가 그 부담을 어떻게 해소하는가
5. Pod Identity 동작 상세
6. 전환 시 운영 편의성 + 정리

Pod 안에서 AWS API를 호출하려면?

- 애플리케이션이 S3·DynamoDB·SQS를 호출하려면 AWS credential이 필요하다
- 그런데 Pod에는 access key를 직접 박지 않는다 — 어디서 자격증명이 오는가?
- 답: AWS SDK의 default credential provider chain이 자동으로 찾아온다
- 이 chain의 동작을 알아야 IRSA·Pod Identity의 차이가 보인다

AWS SDK Default Credential Provider Chain (Java v2 기준)



- **6단계 순차 탐색** — 첫 번째로 찾은 provider에서 종료 ("first wins")
- IRSA = **3rd Web Identity**, Pod Identity = **5th Container**

출처 · sdk-for-java/credentials-chain · eks/pod-id-how-it-works

Python (Boto3) — 12단계 중 핵심 6단계

3 Environment Variables
CI/CD·로컬 셸의 표준 키
`AWS_ACCESS_KEY_ID`
`AWS_SECRET_ACCESS_KEY`

›

4 Assume Role (Profile)
profile의 `role_arn` 기반 chained role
`~/.aws/config`

›

IRSA 진입점

5 Assume Role w/ Web Identity
SA token 파일로 STS `AssumeRoleWithWebIdentity` 호출
`AWS_WEB_IDENTITY_TOKEN_FILE`
`AWS_ROLE_ARN`

›

7 Shared Credentials File
로컬 dev 환경 — Pod 내부에서는 비활성
`~/.aws/credentials`

›

Pod Identity 진입점

11 Container Credentials
노드 loopback의 Pod Identity Agent에 HTTP 호출 (`169.254.170.23`)
`AWS_CONTAINER_CREDENTIALS_FULL_URI`

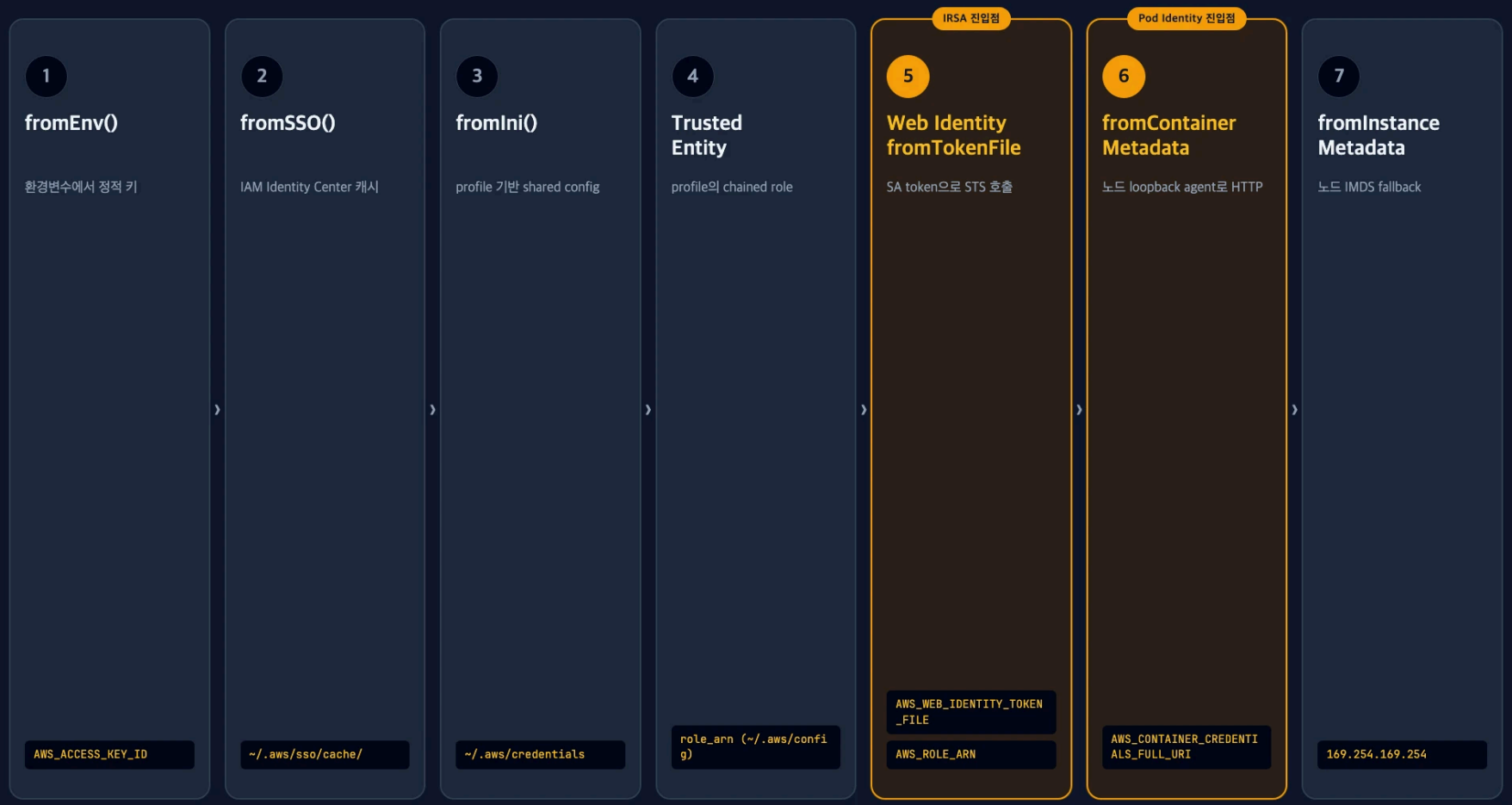
›

12 EC2 Instance Metadata (IMDS)
최후 fallback — 노드 IAM role을 그대로 빌려옴
`http://169.254.169.254/`

- 전체 12단계, 그 중 5번째가 IRSA, 11번째가 Pod Identity (사이에 IAM Identity Center·shared credentials·legacy boto2 등이 있음)
- Java v2와 마찬가지로 web identity가 container provider보다 먼저 평가 — 동일 Pod에 둘 다 있으면 IRSA 우선

출처 · boto3/guide/credentials

JavaScript v3 (Node.js) — 7단계 default chain



- `defaultProvider` 7단계 — IRSA(`fromTokenFile`) 5th, Pod Identity(`fromContainerMetadata`) 6th
- 이름은 다르지만 **순서·환경변수 키는 Java와 동일** — IRSA가 한 단계 먼저

출처 · sdk-for-javascript/v3/setting-credentials-node

Go v2 — 4단계 default chain

IRSA 진입점

1

Environment Variables

정적 키 + web identity token (env 단계가 STS `AssumeRoleWithWebIdentity`까지 처리)

```
AWS_ACCESS_KEY_ID
AWS_WEB_IDENTITY_TOKEN_FILE
```

›

2

Shared Config / Credentials

profile 기반 (Pod 내부에서는 비활성)

```
~/.aws/credentials
~/.aws/config
```

›

Pod Identity 진입점

3

ECS / Container Credentials

노드 loopback agent에 HTTP — Pod Identity Agent 응답

```
AWS_CONTAINER_CREDENTIALS_FULL_URI
```

›

4

EC2 Instance Profile (IMDS)

최후 fallback — 노드 IAM role을 그대로 빌려옴

```
http://169.254.169.254/
```

- env 단계 안에 **web identity token 처리까지 포함** (Java/Boto3/JS는 web identity가 별도 단계)
- 그래도 결과는 같다: **env 안의 IRSA 경로가 container(Pod Identity) 경로보다 먼저**

출처 · sdk-for-go/v2/configure-gosdk

핵심 동작: First Match Wins, 그래서 IRSA가 먼저

- SDK는 chain을 위에서 아래로 순차 평가, **하나라도 매칭되면 멈춘다**
- IRSA 진입점: `AWS_WEB_IDENTITY_TOKEN_FILE` + `AWS_ROLE_ARN` (3rd)
- Pod Identity 진입점: `AWS_CONTAINER_CREDENTIALS_FULL_URI` + `_TOKEN_FILE` (5th)
- Java v2·Boto3·Node.js v3 모두 **web identity가 container provider보다 먼저** (Go v2는 별도 container provider로 동등 효과)

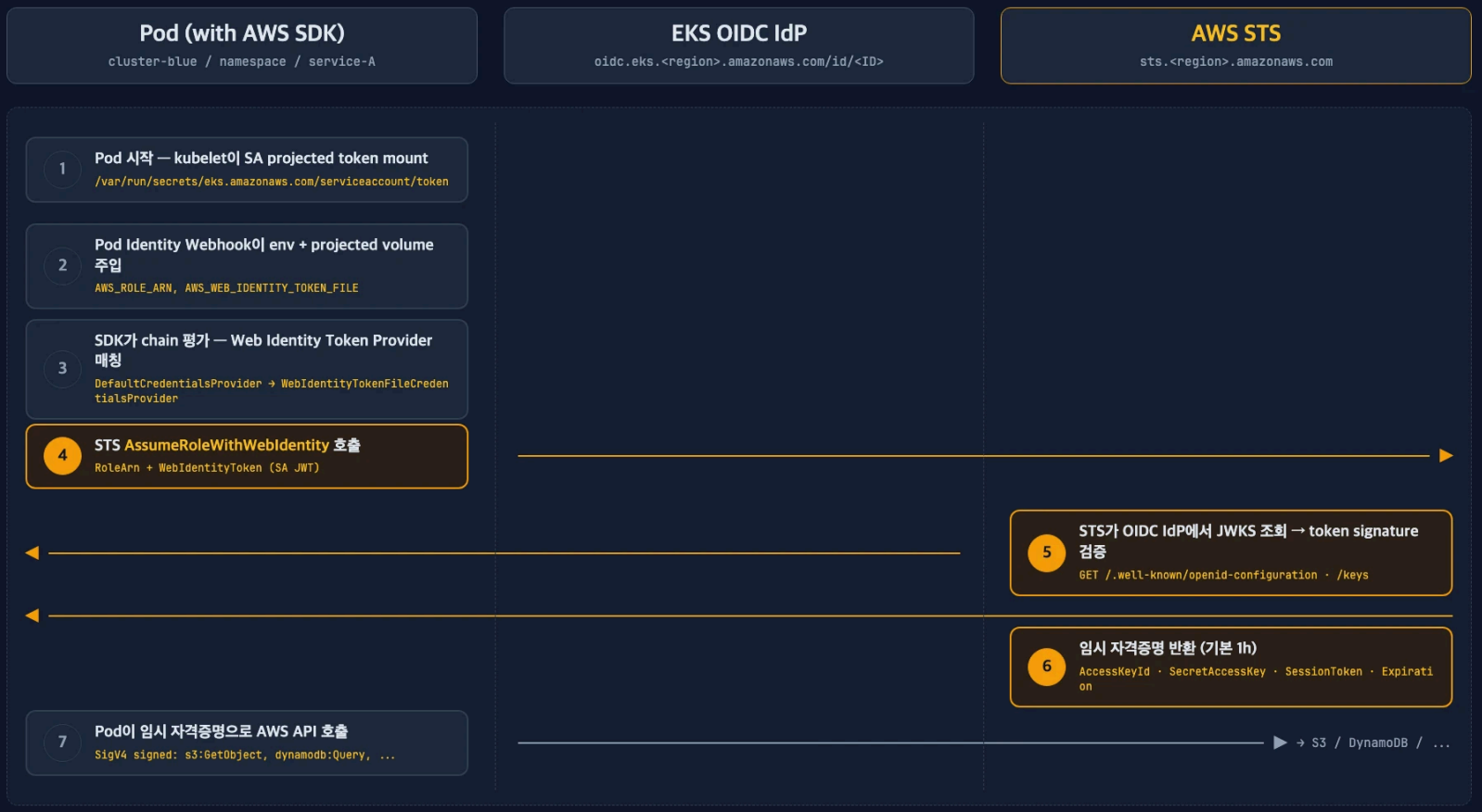
출처 · eks/best-practices/identity-and-access-management · sdkref/feature-container-credentials · boto3/guide/credentials

같은 Pod에 둘 다 있으면? — IRSA 우선 = 마이그레이션 안전망

- AWS 공식: "earlier in the chain… will continue to be used **even if you configure** an EKS Pod Identity association for the same workload"
- 즉, Pod Identity association을 먼저 만들어두고 → IRSA annotation 제거 → 자연스럽게 전환
- AWS는 가능한 경우 **EKS Pod Identity 사용을 권장**
- 그렇다면 IRSA와 Pod Identity는 정확히 어떻게 다른가? — 다음 섹션에서 구조 비교

출처 · eks/userguide/service-accounts

IRSA 구조: SDK가 STS를 직접 호출



- Pod Identity Webhook이 SA annotation을 보고 `AWS_ROLE_ARN` · `AWS_WEB_IDENTITY_TOKEN_FILE` ·projected token volume 주입
- SDK가 token으로 STS `AssumeRoleWithWebIdentity` **직접 호출** → 임시 credential 수신
- 신뢰의 뿌리: **클러스터별 OIDC issuer** + IAM OIDC provider (per-cluster 등록 필요)

출처 · eks/iam-roles-for-service-accounts · STS/AssumeRoleWithWebIdentity

Pod Identity 구조: Agent가 노드에서 대신 받아옴

- 노드 DaemonSet **Pod Identity Agent**가 loopback `169.254.170.23:80`에서 대기
- SDK는 표준 **Container credential provider** slot에서 agent를 호출
- Agent가 EKS Auth API `AssumeRoleForPodIdentity` 호출 → 임시 credential 반환 (**SDK는 STS 미호출**)
- Trust policy Principal은 항상 `pods.eks.amazonaws.com` — **모든 클러스터·계정 동일**

출처 · eks/pod-id-how-it-works · eks/AssumeRoleForPodIdentity

한 장 비교: 신뢰·연결·운영의 5축

IRSA vs EKS Pod Identity — 5축 비교

같은 문제(Pod에 IAM 권한 부여)를 두 방식이 어떻게 다르게 푸는가

	IRSA IAM Roles for Service Accounts (2019~)	EKS Pod Identity pods.eks.amazonaws.com (2023.11~)
1 Trust 메커니즘	**클러스터별 OIDC IdP 등록 필요** `Principal.Federated` = 클러스터 OIDC provider ARN → 신규 클러스터마다 IAM OIDC provider 생성	**단일 service principal로 고정** `Principal.Service` = `pods.eks.amazonaws.com` → 모든 클러스터·계정이 같은 trust 재사용
2 Role 연결 위치	**두 곳 (IAM + K8s) 양쪽에 설정** IAM Role trust policy 의 `sub` / `aud` condition + ServiceAccount `eks.amazonaws.com/role-arn` annotation	**한 곳 (Pod Identity Association)** EKS API `CreatePodIdentityAssociation` 한 번 호출 unique key = (cluster, namespace, serviceAccount)
3 STS quota 사용	사용 **Customer 계정 STS 호출** SDK가 Pod 안에서 직접 `sts:AssumeRoleWithWebIdentity` → Account×Region STS request quota 소비	미사용 **EKS 서비스가 호출** Agent가 `eks-auth:AssumeRoleForPodIdentity` 호출 → Customer STS quota 소비 없음
4 멀티클러스터 trust 갱신	**클러스터 추가마다 모든 Role의 trust 갱신** Trust policy 길이 한도 2,048 → 8,192자 → Role 하나에 trust 관계 보통 ~4개, 증액해도 ~8개	**Trust 변경 불필요 — Association만 추가** Blue/Green 클러스터 전환 시 IAM 변경 0건 새 클러스터에서 Association만 생성
5 의존 컴포넌트	**EKS OIDC IdP + Pod Identity Webhook** Webhook이 SA annotation 보고 env+projected token 주입 (EKS 내장, 별도 add-on 없음)	**Pod Identity Agent DaemonSet (Add-on)** 노드 link-local `169.254.170.23:80` + 노드 IAM Role 권한 (EKS Add-on으로 설치/관리)

Pod Identity는 **trust 정책이 cluster·account에 결합되지 않아 운영 부담이 줄어든다** — 같은 Role을 멀티 클러스터·멀티 계정에 그대로 재사용.

출처: docs.aws.amazon.com/eks/latest/userguide/service-accounts.html · pod-identities.html · pod-id-how-it-works.html · IAM/UserGuide/reference_iam-quotas.html

- **신뢰** OIDC IdP (per-cluster) → service principal (universal)
- **STS quota** customer 소비 → EKS service 호출 (미소비)
- **사전 요건** per-cluster OIDC provider → Agent add-on (Auto Mode 사전 설치)

출처 · CreatePodIdentityAssociation · pod-id-minimum-sdk

그런데 왜 IRSA를 멀티클러스터에서 쓰면 곤란한가?

- IRSA trust policy는 **클러스터의 OIDC provider ARN**을 직접 박는다 — 클러스터별 별도 trust 항목 필요
- 클러스터를 추가할 때마다 **모든 IRSA Role의 trust policy 갱신** 필요

TRUST POLICY의 PRINCIPAL.FEDERATED에 직접 박히는 OIDC PROVIDER ARN

```
arn:aws:iam::123456789012:oidc-provider/oidc.eks.ap-northeast-2.amazonaws.com/id/EXAMPLED539D4633E53DE1B71EXAMPLE
```

앞부분 = 계정·리전 / 뒷부분 `id/<UNIQUE_ID>` = **클러스터별 고유값** — 클러스터 N개 = trust 항목 N줄

출처 · IAM/reference_iam-quotas · eks/best-practices/identity-and-access-management

EKS Cluster Upgrade (Blue/Green)

- `cluster-green` 신규 기동, 워크로드 이전 시작 → 어디선가 401
- 원인: 기존 IRSA Role의 trust policy에 새 OIDC provider 추가 **누락**
- AWS 공식: *"update the IAM role trust policy each time the role is used in a new cluster"*

CLUSTER-BLUE (기존)

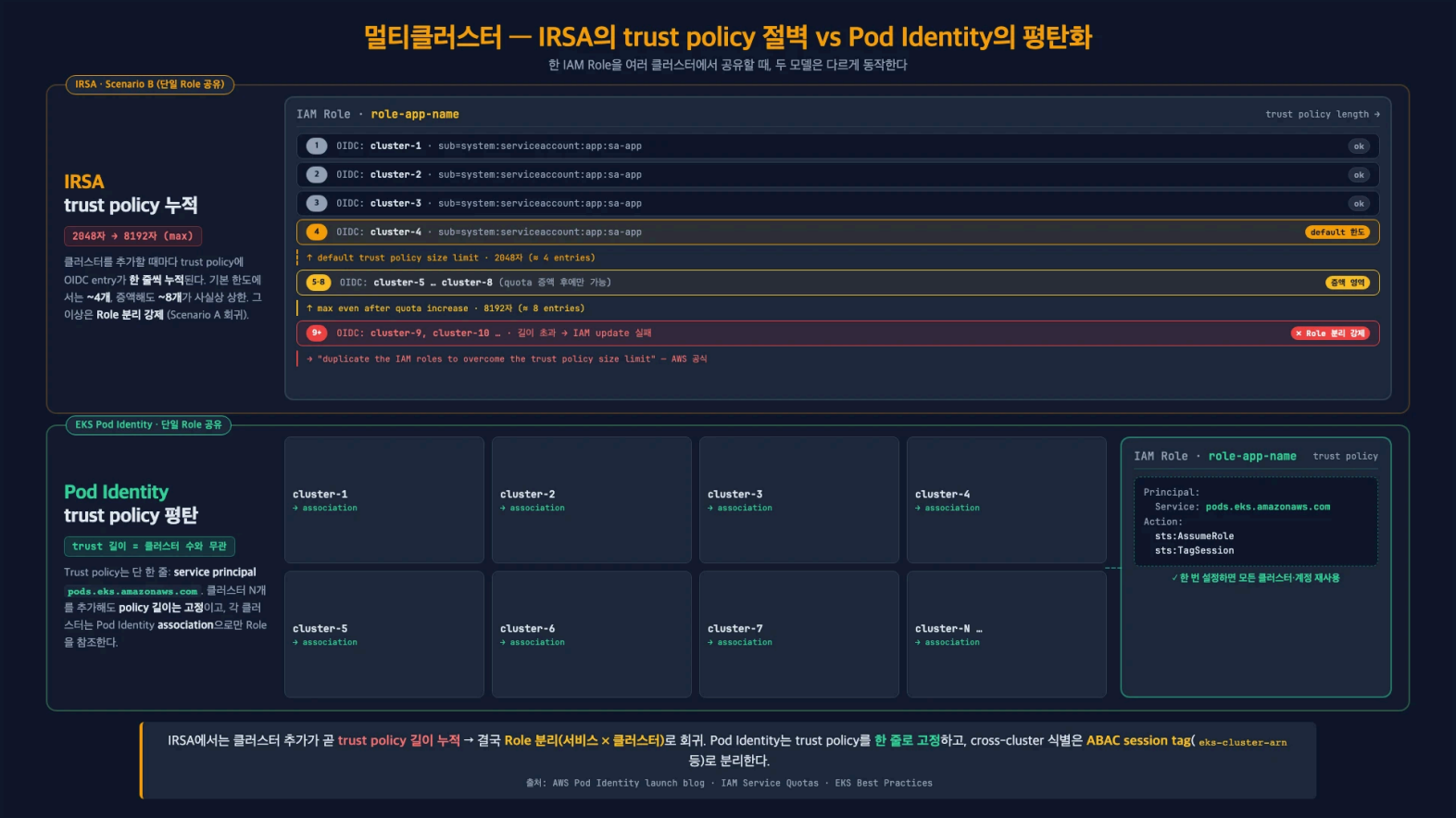
```
arn:aws:iam::123456789012:oidc-provider/oidc.eks.ap-northeast-2.amazonaws.com/id/11111111111111111111111111111111
```

CLUSTER-GREEN (신규) — 모든 IRSA ROLE의 TRUST에 추가 필요

```
arn:aws:iam::123456789012:oidc-provider/oidc.eks.ap-northeast-2.amazonaws.com/id/22222222222222222222222222222222
```

출처 · aws.amazon.com/blogs/containers/...pod-identity · eks/best-practices/identity-and-access-management

그러면 단일 Role + 합친 trust로 가면? — trust policy 길이의 벽



- IAM quota 표: trust policy **default 2,048자, 자동 승인 max 8,192자** (whitespace 제외)
- AWS Pod Identity 공식 blog: *"default 2048자 → typically 4개, 증액해도 typically 최대 8개"*
- AWS EMR docs: *"4,096자 한도 = 단일 Role을 최대 12 EKS 클러스터에서 공유"* (entry 짧은 케이스)
- 결과: 멀티클러스터 5+ 환경에서는 Role 분할 또는 Pod Identity 전환이 강제됨

출처 · IAM/reference_iam-quotas · blogs/containers/pod-identity · EMR-on-EKS/Pod-Identity

장애가 일어났다, 근데 CloudTrail에 없다

- 누락된 trust로 SDK가 `AssumeRoleWithWebIdentity` 호출 → 다음 에러

```
InvalidIdentityToken: No OpenIDConnect provider found in your account
for https://oidc.eks.<region>.amazonaws.com/id/<UNIQUE_ID>
```

- AWS 공식 정책: *"some non-authenticated AWS STS requests might not be logged because they do not meet the minimum expectation of being sufficiently valid to be trusted as a legitimate request"*
- 즉 `InvalidIdentityToken` 같은 client-side 거부는 STS event log에 안 남음
- 운영자 입장에서 가장 답답한 종류의 장애: **"어디서 죽었는지 모르는"** 실패 — Pod log·k8s event로만 추적 가능

출처 · https://repost.aws/knowledge-center/eks-troubleshoot-irsa-errors · https://docs.aws.amazon.com/IAM/latest/UserGuide/cloudtrail-integration.html · https://repost.aws/knowledge-center/iam-oidc-idp-federation

Pod Identity의 답 — Trust는 한 줄, 식별은 Tag로

- Trust policy = 단일 service principal `pods.eks.amazonaws.com` **한 줄**
- 클러스터를 N개 추가해도 trust policy 길이 **변화 없음** — quota 천장 자체가 사라진다
- 클러스터별 식별은 ABAC session tag (`eks-cluster-arn` 등)로 **permission 쪽**에서 처리
- Blue/green 시: 신규 클러스터에서 **association만 만들면 끝** — trust 갱신 불필요
- 그렇다면 Pod Identity는 내부에서 어떻게 동작하는가? — 다음 섹션

출처 · https://docs.aws.amazon.com/eks/latest/best-practices/identity-and-access-management.html

Pod Identity의 trust policy — 단일 service principal

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "pods.eks.amazonaws.com" },
    "Action": ["sts:AssumeRole", "sts:TagSession"]
  }]
}
```

- Principal은 **OIDC URL/account ID에 결합되지 않는** 단일 service principal
- AWS 공식 문구: "*setup the role one time… you don't need to update the role's trust policy each time it is used in a new cluster*"
- 결과: 클러스터가 늘어도 trust policy는 **그대로** — 4~8개 한도 자체가 무의미해짐

멀티클러스터 식별은 ABAC session tag로

- Pod Identity는 assume 시 **6종 session tag를 자동 첨부** (transitive=true)
  - `eks-cluster-arn`, `eks-cluster-name`, `kubernetes-namespace`
  - `kubernetes-service-account`, `kubernetes-pod-name`, `kubernetes-pod-uid`
- cluster/namespace 경계는 **trust policy가 아니라 permission policy의 condition**으로 표현
- 예: `"aws:PrincipalTag/eks-cluster-arn": "arn:aws:eks:ap-northeast-2:123456789012:cluster/cluster-blue"`
- **blue/green 교체** 시 새 클러스터에서 association만 만들면 됨 — trust 갱신 0건

출처 · https://docs.aws.amazon.com/eks/latest/userguide/pod-id-abac.html · https://docs.aws.amazon.com/eks/latest/best-practices/identity-and-access-management.html

Cross-account & OIDC IdP quota — 모두 정리

- **OIDC IdP per-account quota (기본 100, max 700)**: Pod Identity는 IAM OIDC provider를 만들지 않음 → 100개 quota 부담 자체가 사라짐
- **Cross-account**: association의 `targetRoleArn` 파라미터로 EKS가 **role chaining을 자동 수행**, `externalId`로 confused-deputy 방지
- IRSA는 application이 직접 `sts:AssumeRole` chain을 짜야 했지만, Pod Identity는 association 한 번 등록으로 끝
- **다음 섹션에서**: agent가 어떻게 credential을 발급하는지 (`AssumeRoleForPodIdentity` API, 6시간 STS session) 흐름으로 들어갑니다

출처 · https://docs.aws.amazon.com/eks/latest/APIReference/API_PodIdentityAssociation.html

Pod Identity 동작 한 장 — 7-step flow

- SDK는 Container provider slot에서 `AWS_CONTAINER_CREDENTIALS_FULL_URI` 를 읽어 loopback `169.254.170.23` 으로 GET
- 노드의 **Pod Identity Agent**(DaemonSet)가 projected SA token으로 EKS Auth API `AssumeRoleForPodIdentity` 호출
- EKS Auth가 role assumption을 자체 수행 → 임시 credential을 agent → SDK로 전달
- **customer 계정의 STS quota 미소비** — STS 호출 주체가 EKS service

Agent 자체는 어떻게 동작하나

- 노드 DaemonSet + `hostNetwork: true` → loopback `169.254.170.23` (IPv4) / `[fd00:ec2::23]` (IPv6) 의 포트 80·2703에서 listen
- 자기 자신은 노드 IAM Role로 부트스트랩 — managed policy `AmazonEKSWorkerNodePolicy`가 `eks-auth:AssumeRoleForPodIdentity` 액션 포함
- 검증: agent 코드는 Go SDK v2 `LoadDefaultConfig`로 chain 마지막 단계인 IMDS의 EC2 instance role을 사용 — `automountServiceAccountToken: false`라 IRSA 부트스트랩은 불가
- IMDS hop limit=1(EKS Auto Mode 기본값)이어도 agent는 hostNetwork라 노드 ENI에서 직접 IMDS에 도달 — 정상 동작

출처 · https://github.com/aws/eks-pod-identity-agent/blob/d4dc0f3fedd795b26ac88755238867a2110c7460/cmd/server.go#L52-L62 · https://github.com/aws/eks-pod-identity-agent/blob/d4dc0f3fedd795b26ac88755238867a2110c7460/charts/eks-pod-identity-agent/templates/daemonset.yaml · https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-IMDS-existing-instances.html

AssumeRoleForPodIdentity — API 계약

```
POST /clusters/{clusterName}/assume-role-for-pod-identity
Host: eks-auth.<region>.api.aws
Body: { "token": "<projected SA JWT>" }
```

- 응답: `credentials` (SigV4 임시 자격증명: accessKeyId·secretAccessKey·sessionToken·expiration), `assumedRoleUser`, `audience=pods.eks.amazonaws.com`, `subject` (namespace/SA), `podIdentityAssociation` (arn/id)
- Role session name 포맷: `eks-<clusterName>-<podName>-<randomUUID>` → CloudTrail에서 정확히 어떤 Pod인지 추적 가능
- 자동 첨부 session tag 6종(transitive=true): `eks-cluster-name` / `eks-cluster-arn` / `kubernetes-namespace` / `kubernetes-service-account` / `kubernetes-pod-name` / `kubernetes-pod-uid` → policy condition으로 ABAC 작성 가능

Association = (cluster, namespace, serviceAccount)

- Unique key는 세 값의 조합 — **와일드카드/cross-namespace/regex 미지원** (각 필드 단일 string)
- Eventually consistent: API 성공 후 수 초 지연 가능 → high-availability 핫패스에서 association create/update 트리거 금지
- 클러스터당 association 한도: **5,000개**
- 지원 SDK 최소 버전(2023-11~ 릴리스): Java v2 ≥ `2.21.30`, boto3 ≥ `1.34.41`, Go v2 ≥ `release-2023-11-14`, AWS CLI v2 ≥ `2.15.0`, JS v3 ≥ `v3.458.0`
- 동작 메커니즘은 **표준 Container credential provider** slot 그대로 — IRSA 같은 별도 SDK 코드 경로가 아님

운영 시 주의 — Network/Proxy

- **Private subnet**: agent 노드가 EKS Auth API에 도달해야 하므로 `com.amazonaws.<region>.eks-auth` PrivateLink interface endpoint 필요
- **Proxy 환경**: `169.254.170.23` 과 `[fd00:ec2::23]` 을 `no_proxy` / `NO_PROXY` 에 추가 — agent 호출이 외부 프록시로 잘못 라우팅되면 자격증명 획득 실패
- **Cross-account**: association의 `targetRoleArn` 으로 EKS가 두 단계 role assumption 자동 수행 (`externalId` 로 confused-deputy 방지) — 또는 SDK가 transitive session tag를 받아 일반 `sts:AssumeRole` 로 chaining
- **미지원 환경**: AWS Fargate, Windows EC2 노드, EKS Anywhere, Outposts → IRSA 유지 필요
- 다음 섹션: 이 동작 모델이 운영 편의성에 어떻게 이어지는가

출처 · https://docs.aws.amazon.com/eks/latest/userguide/vpc-interface-endpoints.html

운영 부담은 어떻게 줄어드는가 — 4가지

- **Trust 갱신 부담 0건**: trust policy = `Principal.Service: pods.eks.amazonaws.com` 단일 — 클러스터를 추가해도 trust policy를 안 건드림
- **STS quota 미사용**: STS `AssumeRoleWithWebIdentity`를 customer 계정이 직접 호출하지 않음 → EKS service가 대신 호출 → hot pod에서도 안전
- **ABAC session tag 6종**으로 cross-cluster 권한 분리: `eks-cluster-arn` · `kubernetes-namespace` 등 transitive tag로 단일 Role + condition 패턴이 가능
- **Add-on 한 번 설치로 끝**: `eks-pod-identity-agent` add-on 설치 후 association API만 운영 — Auto Mode는 사전 설치

출처 · aws.amazon.com/blogs/containers/...pod-identity · docs.aws.amazon.com/.../pod-id-role.html · docs.aws.amazon.com/.../service-accounts.html · docs.aws.amazon.com/.../pod-id-abac.html · docs.aws.amazon.com/.../pod-id-agent-setup.html

마이그레이션은 단계적으로 — chain precedence가 안전망

- AWS SDK default credential provider chain: **web identity (IRSA, 3rd) > container (Pod Identity, 5th)**
- 동일 Pod에 IRSA annotation + Pod Identity association이 모두 있으면 **IRSA가 그대로 사용됨** — AWS 공식: "credentials earlier in the chain … will continue to be used"
- 안전한 전환 순서:
  i. Pod Identity association 먼저 생성 → 워크로드는 여전히 IRSA로 동작
  ii. `eks.amazonaws.com/role-arn` annotation 제거 + Pod 재시작
  iii. Pod Identity로 자연 전환
- Role의 trust policy에 OIDC provider와 `pods.eks.amazonaws.com`을 **동시에 두는 dual-trust** 패턴으로 롤백 경로도 확보

출처 · docs.aws.amazon.com/sdk-for-java/.../credentials-chain.html · docs.aws.amazon.com/.../pod-id-how-it-works.html · aws.amazon.com/blogs/.../pod-identity

솔직한 한계 — 도입 전 점검 1줄

- **SDK 최소 버전 필요** (2023-11~): Java v2 ≥ `2.21.30`, boto3 ≥ `1.34.41`, Go v2 ≥ `release-2023-11-14`, AWS CLI v2 ≥ `2.15.0` — 오래된 이미지는 우선 업데이트
- **미지원 환경**: AWS Fargate(Linux/Windows 모두), Windows EC2 노드, EKS Anywhere, Outposts → 해당 워크로드는 IRSA 유지
- **Private cluster**: 노드가 EKS Auth API에 도달해야 하므로 `com.amazonaws.<region>.eks-auth` PrivateLink interface endpoint 필요
- 결론: "전체 즉시 전환"이 아니라 **지원 워크로드부터 점진 전환 + 미지원은 IRSA 유지**가 현실적 경로

출처 · docs.aws.amazon.com/.../pod-id-minimum-sdk.html · docs.aws.amazon.com/.../pod-identities.html · docs.aws.amazon.com/.../vpc-interface-endpoints.html

오늘의 take-away 5가지

- **Credential provider chain**을 이해해야 IRSA·Pod Identity의 차이가 보인다 — IRSA = web identity (3rd), Pod Identity = container (5th)
- **IRSA 멀티클러스터 한계는 정량적**: trust policy 2048자(증액 시 8192자) → 한 Role에 trust 관계 ~4개(증액 ~8개)가 사실상 상한
- **Blue/green 클러스터 교체에서 trust 갱신 누락은 CloudTrail로 추적 어려움** — `InvalidIdentityToken`은 client-side로 분류돼 로깅 누락 가능
- **Pod Identity는 trust policy를 단일 service principal로 고정 + STS quota 미사용 + 자동 session tag 6종으로 ABAC**
- **마이그레이션은 chain precedence 덕분에 안전** — association 먼저, annotation 나중

출처 · docs.aws.amazon.com/IAM/.../cloudtrail-integration.html · docs.aws.amazon.com/IAM/.../reference_iam-quotas.html · aws.amazon.com/blogs/.../pod-identity

왜 멀티클러스터에서 Pod Identity인가 — 한 장

- IRSA의 운영 부담은 "OIDC trust 관리"라는 **per-cluster 결합**에서 생긴다
- Pod Identity는 trust 결합을 **per-service(`pods.eks.amazonaws.com`)** 로 옮겨 — 클러스터 수가 늘어도 trust policy는 변하지 않는다
- 결합점이 바뀌면 **운영 토폴로지 자체가 단순해진다** — blue/green·failover·신규 클러스터 추가에서 IAM 작업이 사라진다
- 이것이 30분 발표의 한 줄 결론

출처 · aws.amazon.com/blogs/.../pod-identity · docs.aws.amazon.com/eks/.../identity-and-access-management.html

Thank you — Q&A

김태지 (Ethan)

번개장터 DevSecOps Engineer

발표 자료 (GitHub)
https://github.com/KKamJi98/aws-krug-magok-2026